

Módulo 3. Experiment tracking con MLflow

Curso MLOps/DataOps · ANBAN

José Picón

2026

1. El problema que resuelve el tracking de experimentos

Imagina una sesión típica de modelado sin disciplina. Pruebas un hiperparámetro, sale F1 0,72. Cambias otro, sale 0,74. Cambias otro, 0,71. Vuelves a una combinación anterior, sale 0,76. Lo apuntas en un post-it. Dos horas después, ya no recuerdas qué configuración exacta dio el 0,76, ni qué dataset tenías cargado, ni qué seed pusiste.

Multiplica esto por un equipo de cinco personas durante seis meses y tienes el escenario habitual de cualquier proyecto de machine learning sin una herramienta de tracking.

Un sistema de experiment tracking resuelve tres problemas reales:

1. **Reproducibilidad.** Si guardas parámetros, datos, seed y entorno, puedes recrear el modelo exacto en cualquier momento.
2. **Comparación.** Cuando tienes 30 experimentos, ¿cuál era el bueno y por qué? Sin tracking, vuelves a entrenar todos.
3. **Auditoría.** En sectores regulados (banca, salud, seguros) te van a pedir el log completo de cómo entrenaste cada modelo. Sin tracking, te quedas en blanco.

2. La herramienta principal: MLflow

2.1 Qué es MLflow

MLflow es una plataforma de código abierto para gestionar el ciclo de vida del modelo. Está desarrollada por Databricks pero es completamente independiente y autohospedable. Tiene cuatro componentes:

Componente	Para qué sirve
MLflow Tracking	Registra params, métricas, tags y artefactos de cada ejecución de entrenamiento.
MLflow Models	Estándar para empaquetar modelos, con sus metadatos y entorno.
MLflow Projects	Empaquetado reproducible de proyectos ML. Lo veremos solo de pasada.
MLflow Model Registry	Almacén versionado de modelos con stages (Staging, Production, Archived).

En este curso usamos **Tracking, Models y Registry**. Projects es opcional y se usa menos.

2.2 Vocabulario clave

Aprende este vocabulario porque lo usaremos a partir de ahora:

Término	Significado
Experiment	Carpeta lógica que agrupa runs de un mismo problema. Por ejemplo: <code>heart-failure-classifier</code> .
Run	Una ejecución concreta de entrenamiento. Tiene su propio ID.
Parameter	Valor fijado antes del entrenamiento (hiperparámetro, configuración).
Metric	Valor medido. Puede ser un número final o una serie temporal (por epoch).
Tag	Etiqueta libre clave/valor (ej: <code>git_commit=abc123</code>).
Artifact	Fichero asociado al run (modelo serializado, gráficas, datasets).
Signature	Esquema de inputs/outputs del modelo.
Model Registry	Capa por encima donde un modelo tiene nombre, versión y stage.

Los **stages** de un modelo en el Registry son: `None` (recién registrado), `Staging` (en pruebas), `Production` (sirviendo), `Archived` (retirado).

2.3 Alternativas a MLflow

Antes de comprometerte con MLflow, conoce el panorama:

Herramienta	Punto fuerte	Punto débil
MLflow	Open source, autohospedable, estándar	UI mejorable
Weights & Biases	UI excelente, colaboración fluida	SaaS, on-premises de pago
Neptune	Bueno para deep learning	Comunidad más pequeña
Comet	Muy completo	Comercial
Aim	Open source, rápido	Menos features que MLflow

En empresas con restricciones de datos (banca, salud, RGPD) suele ganar MLflow. En equipos de research donde el presupuesto no es problema, suele ganar Weights & Biases.

2.4 Instalación de MLflow

En el curso MLflow ya está corriendo dentro de un contenedor Docker. No tienes que instalarlo. Pero si quieres usarlo localmente fuera del laboratorio:

macOS, Linux, Windows con WSL

```
pip install mlflow==2.17.2
```

Windows nativo

```
pip install mlflow==2.17.2
```

Verifica:

```
mlflow --version
```

Para arrancar un servidor MLflow local manualmente (no necesario en este curso, MLflow ya corre dentro de Docker):

```
mlflow server --backend-store-uri sqlite:///mlflow.db --host 0.0.0.0 --port  
5000
```

3. Las librerías auxiliares de este módulo

3.1 Scikit-learn (modelo “logreg” y “rf”)

Ya la introdujimos en el módulo 2. Aquí la usamos para entrenar dos modelos clásicos:

- **LogisticRegression**: clasificador lineal, rápido, interpretable.
- **RandomForestClassifier**: conjunto de árboles, mejor precisión, más lento.

Para instalarla:

```
pip install scikit-learn==1.5.2
```

3.2 XGBoost (modelo “xgb”)

XGBoost es una librería especializada en gradient boosting: modelos que construyen árboles de decisión de forma iterativa, cada uno corrigiendo los errores del anterior. Suele dar los mejores resultados en problemas tabulares y es el que casi siempre gana en competiciones de Kaggle.

Para instalarla:

```
pip install xgboost==2.1.2
```

En el lab se instala automáticamente. Si tienes problemas en Mac M1:

```
pip install xgboost --no-binary :all:
```

3.3 Boto3 (cliente S3 para MinIO)

Boto3 es el SDK oficial de AWS para Python. Sirve para hablar con S3, y también con MinIO porque MinIO implementa la API de S3. MLflow usa boto3 por debajo cuando sube artefactos al almacén.

Para instalarla:

```
pip install boto3==1.35.49
```

Lo importante para boto3 son las variables de entorno:

Variable	Valor en este curso
AWS_ACCESS_KEY_ID	minio
AWS_SECRET_ACCESS_KEY	minio12345
MLFLOW_S3_ENDPOINT_URL	http://localhost:9000

4. Antes de la práctica: requisitos

Para hacer este lab necesitas:

1. **Haber completado el lab 1.** Tienes que tener generados los ficheros `labs/lab1_dataops/data/processed/train.parquet` y `test.parquet` . Si no los tienes, vuelve al módulo 2.
2. **El laboratorio del curso arrancado.** Verifica que MLflow responde en `http://localhost:5050` y MinIO en `http://localhost:9001`.
3. **Estar en la carpeta** `labs/lab2_mlflow_dvc/` .

5. Ejercicio: trackear 3 modelos y registrar el mejor

Objetivo del ejercicio: entrenar tres modelos diferentes, comparar sus métricas en MLflow, registrar el ganador en el Model Registry y promocionarlo al stage Staging para que el Lab 3 lo pueda cargar.

5.1 Paso 1 — Exportar las variables de entorno

MLflow y boto3 leen su configuración de variables de entorno. Hay que exportarlas en la terminal donde vas a ejecutar los scripts.

macOS, Linux, Windows con WSL

```
export MLFLOW_TRACKING_URI=http://localhost:5050
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
```

Windows nativo (PowerShell)

```
$env:MLFLOW_TRACKING_URI = "http://localhost:5050"
$env:MLFLOW_S3_ENDPOINT_URL = "http://localhost:9000"
$env:AWS_ACCESS_KEY_ID = "minio"
$env:AWS_SECRET_ACCESS_KEY = "minio12345"
```

Explicación de cada variable:

- **MLFLOW_TRACKING_URI** : dónde está el tracking server. En el curso, en localhost:5050.
- **MLFLOW_S3_ENDPOINT_URL** : dónde está el almacén de artefactos. Como usamos MinIO, le decimos su URL.
- **AWS_ACCESS_KEY_ID** y **AWS_SECRET_ACCESS_KEY** : credenciales para hablar con MinIO via boto3.

Truco práctico: guarda estas líneas en un fichero `setenv.sh` y ejecuta `source setenv.sh` cada vez que abras una terminal nueva.

5.2 Paso 2 — Repasar el script de entrenamiento

Abre `src/train.py`. Las partes clave son:


```

# 1. Conectar con el tracking server
mlflow.set_tracking_uri(os.environ["MLFLOW_TRACKING_URI"])
mlflow.set_experiment("heart-failure-classifier")

# 2. Abrir un run nuevo
with mlflow.start_run(run_name=args.model):
    # 3. Guardar los hiperparámetros
    mlflow.log_params(params)

    # 4. Guardar metadatos como tags
    mlflow.set_tags({
        "git_commit": git_commit(),
        "dataset_version": "lab1-v1",
        "owner": args.owner,
        "framework": type(model).__name__,
    })

    # 5. Entrenar el modelo
    model.fit(X_tr, y_tr)

    # 6. Calcular métricas en test
    metrics = {
        "accuracy": accuracy_score(y_te, y_pred),
        "f1": f1_score(y_te, y_pred),
        "precision": precision_score(y_te, y_pred),
        "recall": recall_score(y_te, y_pred),
        "roc_auc": roc_auc_score(y_te, y_proba),
    }
    mlflow.log_metrics(metrics)

    # 7. Guardar el modelo con su signature
    sig = mlflow.models.infer_signature(X_tr, y_pred)
    mlflow.sklearn.log_model(
        sk_model=model,
        artifact_path="model",
        signature=sig,
        input_example=X_tr.head(3),
        pip_requirements=pip_reqs,
    )

```

Explicación bloque a bloque:

- **set_tracking_uri**: le dice al cliente Python a qué tracking server hablar. Sin esto, MLflow usa una carpeta local.
- **set_experiment**: si el experimento “heart-failure-classifier” no existe, lo crea. Si existe, lo selecciona.
- **start_run**: abre un run nuevo dentro del experimento. El bloque `with` se asegura de cerrarlo aunque el código falle.

- **log_params**: guarda los hiperparámetros del modelo (n_estimators, max_depth, learning_rate, etc.) como diccionario.
- **set_tags**: igual que log_params pero para metadatos arbitrarios. Aquí guardamos el commit de git, la versión del dataset y el usuario que lanzó el run. Cuando audites el modelo en el futuro, estos tags son oro.
- **fit**: entrenamiento estándar de sklearn.
- **log_metrics**: guarda las métricas calculadas en test.
- **log_model**: serializa el modelo, lo sube a MinIO y deja en el run un puntero. La **signature** es el esquema de inputs/outputs: sin esto, cuando luego sirvas el modelo en una API, no sabrías qué columnas debe recibir.

5.3 Paso 3 — Entrenar los tres modelos

Desde `labs/lab2_mlflow_dvc/`, ejecuta los tres entrenamientos en orden:

```
python src/train.py --model logreg
python src/train.py --model rf
python src/train.py --model xgb
```

Cada uno tarda entre 30 y 60 segundos. Al final de cada uno verás una línea como:

```
==      rf == accuracy=0.8612  f1=0.6877  precision=0.7561  recall=0.6306
roc_auc=0.9120
```

5.4 Paso 4 — Comparar los runs en la UI

Abre <http://localhost:5050> en el navegador. En la barra lateral izquierda, click en el experimento **heart-failure-classifier**. Verás los tres runs.

Para compararlos:

1. Marca las tres casillas a la izquierda de los runs.
2. Pulsa el botón **Compare**.
3. Aparece una vista con cuatro paneles: parámetros, métricas, coordenadas paralelas y dispersión.
4. En el panel de métricas, ordena por F1 descendente. El ganador suele ser **xgb** o **rf**.

5.5 Paso 5 — Registrar el modelo ganador

Tienes dos formas de registrar el modelo: por la UI o por código. Vamos a hacerlo por código, que es lo que harías en un pipeline real.

```
python src/register_best.py --experiment heart-failure-classifier --metric f1
--name heart-failure-clf
```

El script `register_best.py`:

1. Busca el run del experimento `heart-failure-classifier` con mayor F1.
2. Llama a `mlflow.register_model()` con el path del modelo de ese run.
3. Crea una nueva versión del modelo `heart-failure-clf` en el Registry.

Recarga `http://localhost:5050` y entra en la pestaña **Models** arriba a la derecha. Verás `heart-failure-clf` con su versión 1.

5.6 Paso 6 — Promocionar a Staging

En el Registry, las versiones empiezan en stage `None`. Para promocionarla:

```
python src/promote.py --name heart-failure-clf --version 1 --stage Staging
```

Recarga la UI. La versión 1 ahora está en **Staging**.

¿Para qué sirve esto? Cuando en el Lab 3 sirvamos el modelo con FastAPI, la API apuntará a `models:/heart-failure-clf/Staging`. Si mañana quieres cambiar el modelo que sirve en producción, solo tienes que promocionar otro y la API lo recoge en su próxima recarga. **No hay que volver a desplegar código.**

5.7 Paso 7 — Cargar el modelo desde Python

Para terminar, verifiquemos que el modelo es accesible desde Python. Lanza un Python interactivo:

```
python
```

Y ejecuta:

```
import os
import mlflow.pyfunc

os.environ["MLFLOW_TRACKING_URI"] = "http://localhost:5050"
os.environ["MLFLOW_S3_ENDPOINT_URL"] = "http://localhost:9000"
os.environ["AWS_ACCESS_KEY_ID"] = "minio"
os.environ["AWS_SECRET_ACCESS_KEY"] = "minio12345"

m = mlflow.pyfunc.load_model("models:/heart-failure-clf/Staging")

print("Run ID:", m.metadata.run_id)
print("Inputs:", len(m.metadata.signature.inputs.inputs), "columnas")
```

Verás algo así:

```
Run ID: 1944ae332b7249a9874dd98fd20c381a
Inputs: 95 columns
```

Esa única línea (`models:/heart-failure-clf/Staging`) es todo lo que necesita la API para cargar el modelo correcto, con su esquema completo y trazabilidad al run que lo entrenó. Eso es el Registry en su esencia.

Sal del Python con `exit()` o `Ctrl+D`.

6. Funciones de MLflow explicadas una a una

Función	Qué hace
<code>mlflow.set_tracking_uri(url)</code>	Indica al cliente Python dónde está el tracking server.
<code>mlflow.set_experiment(name)</code>	Selecciona el experimento. Lo crea si no existe.
<code>mlflow.start_run(run_name=name)</code>	Abre un run nuevo. Se usa como context manager (<code>with</code>).
<code>mlflow.log_param(key, value)</code>	Guarda un único hiperparámetro.
<code>mlflow.log_params(dict)</code>	Guarda varios hiperparámetros a la vez.
<code>mlflow.log_metric(key, value)</code>	Guarda una métrica. Si la llamas varias veces, se guarda como serie temporal.
<code>mlflow.log_metrics(dict)</code>	Igual pero con varias métricas.
<code>mlflow.set_tag(key, value)</code>	Añade un tag al run.
<code>mlflow.set_tags(dict)</code>	Varios tags a la vez.
<code>mlflow.sklearn.log_model(sk_model, artifact_path, signature, input_example, pip_requirements)</code>	Serializa el modelo sklearn, lo sube y registra en el run.
<code>mlflow.pyfunc.load_model(uri)</code>	Carga un modelo desde una URI tipo <code>models:/nombre/Staging</code> o <code>runs:/run_id/model</code> .
<code>mlflow.MlflowClient().register_model(model_uri, name)</code>	Crea una nueva versión de un modelo en el Registry.
<code>mlflow.MlflowClient().transition_model_version_stage(name, version, stage)</code>	Mueve una versión entre stages (Staging, Production, Archived).

7. Métricas de clasificación explicadas

Hemos calculado cinco métricas en cada run. Conviene entender qué mide cada una:

Métrica	Qué mide
Accuracy	Porcentaje de predicciones correctas sobre el total. Engaña en datasets desequilibrados.
Precision	De los que el modelo dijo “positivo”, ¿cuántos lo eran realmente?
Recall	De los que eran “positivo” en la realidad, ¿cuántos detectó el modelo?
F1	Media armónica de precision y recall. Buena métrica general.
ROC-AUC	Capacidad del modelo de ordenar bien (independiente del umbral). Va de 0,5 (aleatorio) a 1,0 (perfecto).

En un problema de detección de fraude o de morosidad te importa **recall** (no perderte casos). En un problema de spam te importa **precision** (no marcar correo legítimo). El F1 es el equilibrio.

8. Checklist de cierre

- ¿Has entrenado tres modelos y aparecen los tres runs en la UI de MLflow?
- ¿Tienes tags `git_commit`, `dataset_version`, `owner` y `framework` en cada run?
- ¿Tienes un modelo logueado con signature en cada run?
- ¿Existe el modelo `heart-failure-clf` versión 1 en el Registry, en stage `Staging`?
- ¿Has podido cargar el modelo desde Python con `models:/heart-failure-clf/Staging`?

Si todo es sí, pasas al módulo 4 (serving).

9. Para profundizar

- **MLflow documentation:** <https://mlflow.org/docs/latest/index.html>. La sección “Tutorial” es muy buena.
- **Designing Machine Learning Systems**, Chip Huyen (O’Reilly, 2022). Capítulo de experiment tracking.
- **MLflow vs Weights & Biases:** comparativa rápida en <https://wandb.ai/site/articles/mlflow-vs-wandb>.